Multi-Action Tangled Program Graphs for Multi-Task Reinforcement Learning with Continuous Control

Quentin Vacher[1], Nicolas Beuve[1], Mickaël Dardaillon[1], and Karol Desnos[1]

Univ Rennes, INSA Rennes, CNRS, IETR – UMR 6164, F-35000 Rennes, France
first.last@insa-rennes.fr

Abstract. Over the past few decades, machine learning has been widely used to learn complex tasks. Reinforcement Learning (RL), inspired by human behavior, is a great example, as it involves developing specific behaviours for specific tasks. To further challenge algorithms, Multi-Task RL (MTRL) environments have been introduced, requiring a single model to learn multiple behaviors.
The Tangled Program Graph (TPG) algorithm is a Genetic Programming (GP) algorithm designed for discrete MTRL environments. Recently, the MAPLE algorithm has been proposed, as another GP algorithm that achieves high results in single task continuous RL environments. A variation of the TPG is proposed alongside MAPLE, named Multi-Action TPG (MATPG) that aggregates MAPLE agents, and creates a control flow to activate them. Initially tested on single task RL environments only, MATPG achieved similar results to MAPLE.
In this work, we present a new benchmark based on the MuJoCo *Half Cheetah* from Gymnasium. This benchmark features five distinct obstacles that are randomly positioned in front of the agent, each of which demands a unique behavior. This benchmark serves as a use case for MATPG, to prove its ability as a GP solution for continuous MTRL environments. Our experiments demonstrate its superiority in this multi-task use case when combined with lexicase selection. Furthermore, we examine the interpretability of the evolved graph, revealing that the decision flow of the model is fully interpretable.

Keywords: Multi-task Learning · Reinforcement Learning · Tangled Program Graphs · Continuous Control

1 Introduction

Intelligence can be measured by the ability to solve diverse tasks. Artificial intelligence has explored learning multiple tasks with single models, particularly in RL, where agents reason over long-term consequences, solving the temporal credit assignment. In MTRL, this challenge extends across tasks, requiring deep environmental understanding.

Several MTRL benchmarks capture these challenges [11]. In continuous control, many use the Multi-Joint dynamics with Contact (MuJoCo) suite [24] from Gymnasium [25], by modifying body parts or controlling multiple agents [33]. Most solutions employ deep learning, specifically Deep RL (DRL) [8,9,20,23], using transfer learning to share knowledge. While DRL performs well, its architectures are complex and opaque, producing large, uninterpretable models [4,12,18].

Alternative machine learning architectures, such as GP, solve RL tasks while producing compact and interpretable solutions. Algorithms like Linear GP (LGP) [2], Tree-Based GP (TGP) [15], and Cartesian GP (CGP) [19] handle continuous control, while TPG [13] excels in discrete MTRL [14]. In [29], MATPG was proposed as a continuous adaptation of TPG. However, it does not outperform its simplified version, Multi-Action Programs through Linear Evolution (MAPLE), on single continuous control tasks.

We evaluate the effectiveness of MATPG in continuous-control MTRL settings, comparing it against its simplified version, MAPLE, under both tournament and lexicase selection [7,10]. Tournament selection is the default method for MAPLE and MATPG in [29], while lexicase selection, originally designed for multi-task learning in classification and regression, encourages task-specific elites. Our experiments demonstrate that both MATPG and MAPLE with lexicase selection outperform tournament selection in producing agents capable of solving individual tasks. Furthermore, MATPG proves highly effective in continuous-control MTRL environments, achieving significantly better performance than the other three configurations when evaluated on all tasks simultaneously with lexicase selection.

To assess these algorithms, we designed a MTRL benchmark based on the MuJoCo *Half Cheetah* environment, shown in Figure 1, extended with five independent obstacles that an agent must overcome. This benchmark is modular, meaning that learning one task does not enable the agent to overcome another without explicit learning. It has been verified that the five obstacles can be learned independently by a single agent, making it a robust testbed for evaluating MATPG and MAPLE. All the code and results used in this study are available on [28].

Section 2 reviews prior work on MTRL environments and GP algorithms applied to RL. Section 3 provides a detailed description of the MATPG algorithm. Our proposed benchmark, based on a customized *Half Cheetah*, is presented in Section 4, followed by the experimental setup in Section 5, including the selection algorithms and parameter settings. Results are reported in Section 6, and Section 7 explores the policy of the best MATPG agent, highlighting its interpretability. Finally, Section 8 concludes the paper and discusses the findings.

2 Related Work

To motivate the need for a new MTRL benchmark, we provide a review of existing benchmarks in RL in Section 2.1. In this work, we challenge the proposed benchmark using the MATPG algorithm, which allows explicit reasoning over

multiple tasks. A complementary review of GP algorithms applied to RL is presented in Section 2.2, leading up to the development of MATPG.

2.1 Multi-task Reinforcement Learning for Continuous Control

Recent advances in Reinforcement Learning (RL) span diverse domains including robotics, games, autonomous systems, and NLP [31]. This has driven development of DRL algorithms for continuous control like Soft Actor-Critic (SAC) [9], Proximal Policy Optimization (PPO) [23], and Twin Delayed Deep Deterministic (TD3) [8]. Continuous control benchmarks, primarily in Gymnasium [25], often use the MuJoCo physics engine [24]. This benchmark comprises different robotic tasks that aim to reproduce real-world physics simulations. Thus, the MuJoCo benchmark is highly effective for scalable, realistic, single-task continuous control with RL.

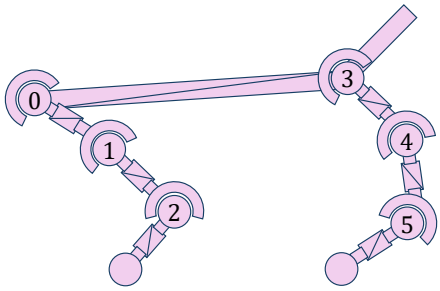


Fig. 1: Scheme of the *Half Cheetah* environments from the MuJoCo suite of Gymnasium.

To evaluate multi-task capabilities, benchmarks requiring simultaneous task learning have emerged, leading to specialized MTRL algorithms. IMPALA [6] learns 57 Arcade Learning Environment (ALE) games with discrete actions, while continuous control MTRL uses MuJoCo, modifying *Half Cheetah* morphology [33,11] or learning six tasks simultaneously with single-task performance.

However, DRL suffers from interpretability issues due to its Deep Learning (DL) architecture [18]. For instance, a standard MTRL network with two 400-unit layers exceeds 100k multiply–accumulate (MAC) operations, complicating understanding and increasing complexity [33]. Alternative architectures like GP address this by evolving compact, interpretable policies.

2.2 Genetic Programming for Reinforcement Learning

As in many other areas, Genetic Programming (GP) [15] has been used to learn RL tasks. GP can significantly improve the interpretability [4] and reduce the computational complexity of a solution by orders of magnitude [27] compared to DRL. In [30], the LGP [2] and TGP [15] representations are used to learn different discrete tasks in the Gymansium library [25] such as the *CartPole* or the *Lunar Lander*, as well as some continuous tasks, such as *Hopper* or the *Bidepal Walker*. In [21], LGP ant CGP [19] representations are trained independently on eight MuJoCo environments. Compared to state-of-the-art DRL algorithms, the GP solutions achieve equivalent or better results on the tasks with three or fewer actions. However, when the number of actions increases, DRL solutions become superior.

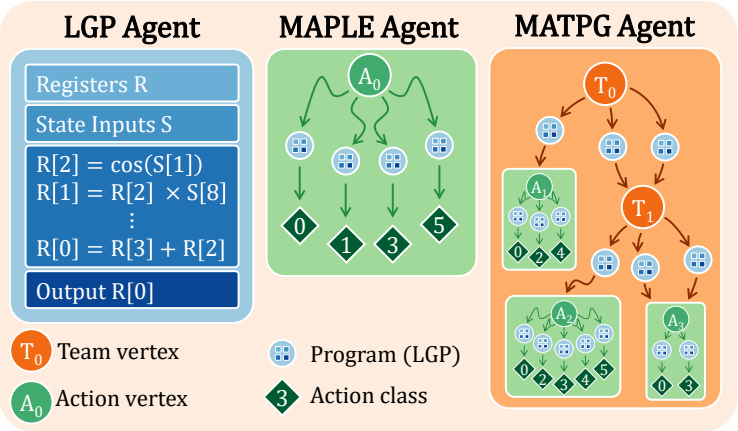


Fig. 2: Examples of LGP, MAPLE and MATPG agents for an environment with six continuous actions. An action vertex uses all the output values of its programs to take the continuous actions, while a team vertex only follow the path of the program outputting the highest value.

For discrete MTRL, TPG evolved a graph of LGP programs, initially achieving Deep Q-Network (DQN)-level performance on ALE with single Atari game, and later learning multiple Atari games with one model [13,14]. It was subsequently extended to continuous RL by replacing discrete actions with an LGP program outputting continuous values [1]. To scale to many actions, two related approaches are proposed in [29]: MAPLE and MATPG employing multiple LGP programs, one per action. MAPLE is a flat multi-program policy, while MATPG integrates this into the hierarchical structure of TPG.

On MuJoCo benchmarks, MAPLE generally outperformed the continuous TPG and matched MATPG, but with a smaller model size. The authors note that standard MuJoCo tasks do not leverage key strength of MATPG: multi-task learning [29]. This paper therefore investigates a dedicated MTRL use case for MATPG. Before doing so, the MATPG algorithm is presented with more details in the next section.

3 Background: Multi-Action Tangled Program Graphs

Semantics of MATPG. MATPG [13,29] consists of three main entities forming a direct graph: teams, actions, and programs. Note that MAPLE is a specific case of MATPG without teams. Figure 2 shows an example of LGP, MAPLE and MATPG agent. Each team and action vertex contains multiple edges with programs corresponding to independent LGP. Team programs point to another vertex, while action programs point to an action class. Execution begins at a designated team, called root team. This team activates all its programs with the

same state inputs from the observation of the environment. The destination with the highest program output is followed. If it points to another team, the process repeats. If it points to an action vertex, that vertex activates its programs. Each program outputs a continuous value defining the final action for its class. Missing action classes use a default action. The actions taken are then sent to the environments.

Mutation process. The graph structure evolves dynamically. Each generation creates a fixed number of roots by duplicating and mutating the survivors of natural selection process. Mutation can add or remove edges, or mutate existing ones. When mutating an edge, either its program or destination can be mutated. If the edge led to an action vertex, that vertex can be mutated instead of the program.

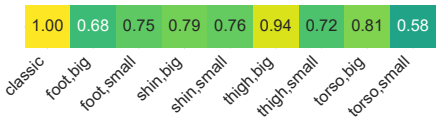


Fig. 3: Scores achieved by maple agents trained on classic *Half Cheetah* but tested with modified body size. Scores are normalized by the unmodified score of 3622.

An action vertex is a MAPLE agent which is mutated in a similar fashion to team vertex, with the additional constraint that than any action class can appear only once in the action class edges.

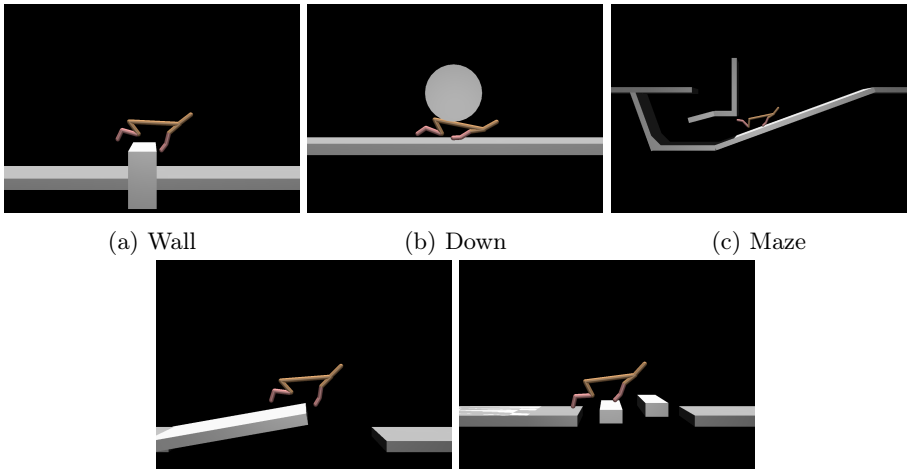
Sub-population evolution. MATPG uses two sub-populations: one third MATPG agents and two thirds independent MAPLE agents. During the evolution of an MATPG agent, edge destinations can be changed to MAPLE agents. This approach was used in [29], where authors claimed MATPG alone struggles to learn due to high complexity. Therefore, the learning is divided with MAPLE agents learning behaviors, and MATPG agents learning how to combine them.

As mentioned in Section 2.2, MATPG did not surpass MAPLE in [29]. The hypothesis is that the original MuJoCo benchmark lacks multi-task requirements where MATPG should excel. Thus, a MTRL application is needed to measure the efficiency of MATPG.

4 Use Case: Customized Half Cheetah

To measure MATPG efficiency, the benchmark must be an MTRL environment. Tasks must be challenging enough to be independent, where mastering one doesn't imply success in others without further training. They must also be accessible enough for MAPLE agents to succeed individually. In this section, we introduce a new benchmark for MTRL in continuous control, using the *Half Cheetah* from the MuJoCo suite. Inspired by [26], it comprises five distinct obstacle tasks that the *Half Cheetah* must overcome.

The two benchmarks discussed in Section 2.1 fail to meet our requirements. The *Half Cheetah* body-size variation benchmark exhibits excessive task correlation, as evidenced by the performance of a MAPLE agent trained without any



(a) Wall (b) Down (c) Maze

(d) Jump (e) Stairs

Fig. 4: The different obstacles used in the benchmark for the *Half Cheetah*.

variations on *Half Cheetah* and tested on this benchmark, see Figure 3. The benchmark requiring multi-task learning across the MuJoCo suite is unsuitable due to its potential excessive complexity and the need for task-specific, independent knowledge. Additionally, unlike real-world multi-task control where tasks involve the same body in different environments, multiple MuJoCo environments introduce distinct input and action spaces, and body morphologies.

Our proposed benchmark strikes a balance: the tasks are independent enough to make achieving a high score challenging for an agent encountering them for the first time, yet they retain similarity through shared *Half Cheetah* behaviors. This setup also enables evaluation in single episodes featuring sequential obstacles.

4.1 The obstacles

Figure 4 shows the different obstacles used in the benchmark:

- **Wall:** A wall that the *Half Cheetah* needs to climb over.
- **Down:** A cylinder that creates a small tunnel through which the *Half Cheetah* can only pass by getting down.
- **Maze:** A maze where the *Half Cheetah* needs to fall, then go backward, and finally climb a slope. The objective is not the slope itself, so the friction on the slope is maximized to help the *Half Cheetah*.
- **Jump:** As with the previous obstacle, the objective is the jump itself, so the friction on the small slope beforehand is increased to help the *Half Cheetah* gain speed.
- **Stairs:** A set of stairs with holes that the *Half Cheetah* needs to avoid.

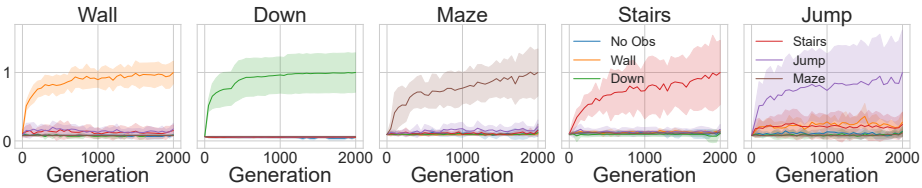


Fig. 5: Validation score achieve on every obstacle, by populations of MAPLE agents trained on each obstacle individually and one on no obstacle. The main curve is the mean score over five seeds, and the shaded area is the standard deviation.

4.2 The environment

The environment is based on the *Half Cheetah-v5* from Gymnasium [25]. In this environment, the agent performs six actions, applying torques to each of the six joints of the *Half Cheetah*, as shown in Figure 1. To learn these actions, the sensor inputs are composed of 17 values representing position and velocity information. The default length of an episode is 1000 steps. The objective is to move the *Half Cheetah* as far to the right as possible.

To implement obstacles, the environment is divided into 10-meter sections. When the *Half Cheetah* enters a section, the center of the obstacle is randomly placed between the fourth and fifth meter. This randomness of the position is important to ensure no overfitting. Two observations are added to the standard 17: the identifier of the current obstacle and its center position. While in the obstacle section, the distance to this center is provided, even if negative.

Two additional modifications from the original MuJoCo task were made. First, to prevent the *Half Cheetah* from falling on its back, the episode ends if the z-orientation of the torso leaves $[-2.5,\ 2.5]$, to avoid undesirable behavior, particularly in the *Maze*. Second change is on the reward, originally divided in two sub rewards. The forward reward, rewarding the agent for going as fast as possible to the right. The control cost, penalizing the agent for applying high torque to the agent motors. While control cost makes sense for pure speed maximization, it becomes less necessary here and can create strong local minima if tasks aren't learned quickly, thus only the forward reward is used.

4.3 Initial results

Based on the experimental setup detailed later in Section 5.2, MAPLE agents are trained on each obstacle individually using five seeds. Figure 5 presents the results for each task. At each validation step, agents are tested on all tasks to assess untrained learning. The results clearly show they do not learn untrained tasks. This observation also holds for populations trained without obstacles.

To further validate, the best agents from each training run are tested ten times on each obstacle. The correlation matrix in Figure 6 summarizes these results. Since obstacles like *Maze* require more time steps than *Jump*, raw scores aren't directly comparable. The matrix uses normalized scores, dividing each score by the average achieved by populations trained on that specific obstacle. The correlation matrix reveals complete task independence: agents fail on unseen tasks. This contrasts with Figure 3, where such generalization does occur.

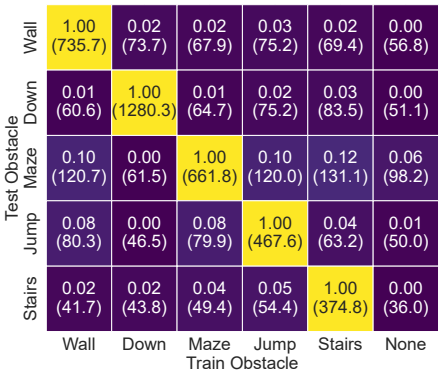


Fig. 6: Correlation matrix of the nomalized score. The x-axis is the obstacle the MAPLE agents have been trained on, while the y-axis is the normalized scores reached on each obstacle. The normalized score presented is obtained by dividing the score of the cell by the score of the trained obstacle, the original score is shown within the brackets

Thanks to this training, a coefficient can be calculated for each obstacle to indicate its level of difficulty. Since each task is independent, the difficulty cannot be measure solely by the final score. It is more relevant to base the difficulty on how quickly the task can be learned. Additionally, a higher instability between seeds indicates a higher difficulty. The metric is based on the Area Under the Curve (AUC) of the training [17], corresponding to the following equation:

$$\text{For each task } t, \quad \text{AUC}_t = \frac{\sum_{i=1}^{n}(\mu_{i,t} - \sigma_{i,t})}{\mu_{n,t}} \tag{1}$$

Where n is the number of generations, $\mu_{i,t}$ and $\sigma_{i,t}$ represent the average and standard deviation of the score at generation i for task t. The AUC corresponds to the area under the curve below the shaded region in Figure 5. For obstacles *Wall*, *Down*, *Maze*, *Jump*, and *Stairs*, the AUC values are 29.7, 26.0, 20.6, 16.3, and 10.4, respectively. Normalizing by the highest score yields coefficients of 1.00, 0.88, 0.69, 0.55, and 0.35, where higher values indicate easier tasks. This ranking is supported by one seed failing entirely on the *Stairs* obstacle.

In addition to training with MAPLE, experiments used the state-of-the-art DRL algorithm SAC [9] on the same obstacles. Each obstacle was trained with five independent repetitions, with results shown in Figure 7, normalized by MAPLE scores . SAC consistently learned the *Down* obstacle quickly. *Maze* and *Stairs* generally required over one million steps, except one *Maze* seed learning faster. For *Jump*, only one seed succeeded, and none on *Wall*.

Although SAC learns efficient behaviors for *Down* and *Maze*, it struggles with new strategies needed for obstacles like *Wall*. While not designed for MTRL, when trained across all obstacles simultaneously, it solved only two versus three when trained separately, supporting the independence of the obstacles.

5 Experimental Setup

This section describes the experimental setup of the study. The parametrization mainly follows [29]. Section 5.1 focuses on selection methods, the main change from the original MATPG paper, presenting both tournament and lexicase selection. The parametrization is then detailed in Section 5.2.

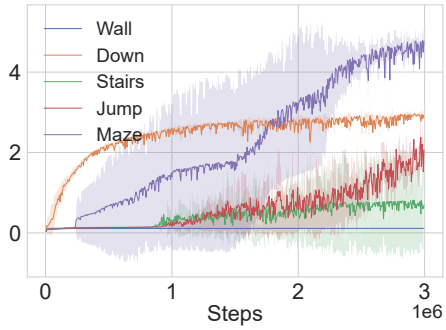


Fig. 7: Validation score achieve on every obstacle, by the SAC. The main curve is the mean score over five seeds, and the shaded area is the standard deviation.

5.1 Selection methods

Tournament selection [7] is the algorithm originally used in MATPG. It is widely used and particularly suited for single-task environments. The process for each generation is as follows:

- A small proportion of top-performing agents, called *elites*, are copied directly to the next generation.
- The remaining population is randomly divided into fixed-size subsets. From each subset, the best agent, the winner is selected for reproduction.
- Winners undergo mutation and crossover to generate offspring.
- The next generation consists of elites and the new offspring.

Lexicase selection [10] was originally designed for classification tasks to promote agents capable of learning multiple classes. The process to select each survivor is:

- A random task order is generated.
- For each task in order, only the top-performing agents are retained.
- After all tasks, if one agent remains, it is selected. If multiple remain, one is chosen randomly.

This repeats until the desired number of survivors is achieved. Survivors generate offspring for the next generation, while others are discarded.

Lexicase selection was proposed for discrete integer scores where top performers often tie. Epsilon-lexicase [16] is a variant for regression problems with continuous scores, where exact equality is rare. In epsilon-lexicase, selection is relaxed: all agents with scores in $[\max(score) \times \epsilon,\ \max(score)]$ are retained, where ϵ is computed from median absolute deviation. This allows more flexible selection. In our context, epsilon-lexicase is more appropriate due to continuous scores. However, since scores are very high in RL, we multiply the epsilon coefficient by 0.1 to limit its impact.

5.2 Parametrization

To achieve the same solution as in [29], we use the same training framework based on the open source Gegelati library [5]. This C++ library originally implemented TPG and was extended for MAPLE.

We use the same hyperparameters as [29] for MATPG training. The instruction set is $H = \{+, -, \times, \div, \text{max}, \text{exp}, \text{log}, \text{sin}, \text{cos}, \text{tan}, \text{modulo}\}$. Instructions include constants multiplied by their results, with constants mutation based on the mutation in AutoML-Zero [22]. For MAPLE agents on single tasks in Section 4.3, identical hyperparameters with tournament selection are used: 1000 agents and MAPLE proportion of 1. For MATPG, values are set to 1500 agents with 0.67 MAPLE proportion, leading to 1000 MAPLE and 500 MATPG agents.

The number of episodes per generation indicates how many times the process needs to be repeated to achieve an accurate score. A value of three per generation is chosen for training, leading to fifteen episodes when five obstacles are used.

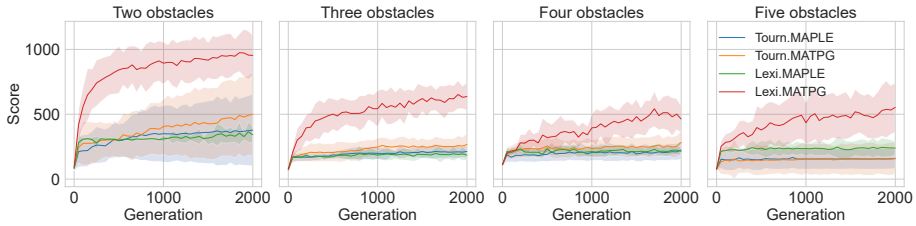
6 Experiments

In this Section, the evaluation of MATPG is done based on the experimental setup established in Section 5. An ablation study is done to evaluate the performances. In a way, MAPLE is a simplification of MATPG, thus both algorithms are tested with either tournament and lexicase selection. Videos showing the behaviors are available in the supplementary materials.

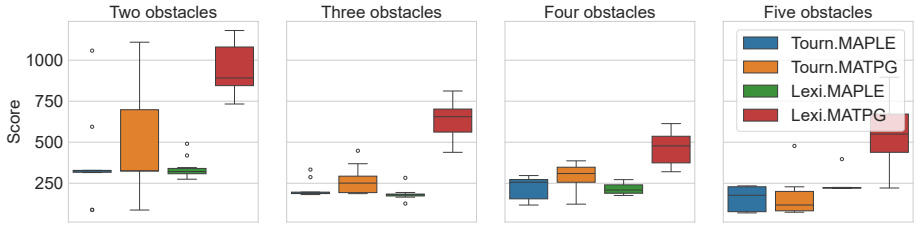
We show through an ablation study the results of MATPG compared to MAPLE, with either the lexicase or the tournament selection. In addition to evaluating the scalability of the different solutions in overcoming obstacles, the tests are conducted not only on the full set of five obstacles but also on subsets comprising two, three, and four obstacles. The selection of tasks is based on their difficulty, as computed in Section 4.3. Specifically, for the two-obstacle scenario, the two easiest obstacles are used for training. This approach is incrementally extended up to the full set of five obstacles.

Figure 8 presents the training performance of the four evaluated approaches across environments with two to five obstacles. In all scenarios, MATPG combined with lexicase selection consistently outperforms both MATPG with tournament selection and MAPLE under either selection method. For the two-obstacle environment, MATPG with tournament selection shows slightly better performance than MAPLE, though the difference is not statistically significant. As the number of obstacles increases, the performance of MATPG with tournament selection aligns closely with that of MAPLE. These results indicate that the original MATPG approach [29] alone struggles to solve MTRL tasks with independent objectives. However, integrating lexicase selection enables MATPG to effectively perform multi-task learning in continuous control environments.

Statistical significance is assessed following prior work [3], using a two-tailed Welch's t-test [32], which is appropriate for small samples with unequal variances. With $N = 10$ independent runs per method, we apply a Bonferroni correction



(a) Results over generations. The main curve is the mean score over ten seeds, and the shaded area is the standard deviation.



(b) Box plots showing the performances of the best agent of each seed for each solution at the end of the training.

Fig. 8: Validation score achieve on average on five episodes with all trained obstacles showing in random order. Tourn. and Lexi. respectively mean tournament and lexicase selection.

to control the family-wise error rate, yielding an adjusted significance threshold of $\alpha_{\text{corr}} = 0.005$. Practical significance is measured using Cohen's d, and, we consider $d > 2$ to indicate a sufficiently large effect under limited-run settings. The results in Table 2 report the p-values and corresponding Cohen's d for Lexicase MATPG against all solutions. Across all comparisons, we observe $p < 0.005$ and $d > 2$, demonstrating statistically significant and practically substantial improvements. Therefore, the performance gains are unlikely to be due to random variation.

Table 1 presents the results obtained by each solution for individual obstacles. The first part of the table reports, for each training, the best score achieved by any agent within its entire population, that is, each score can be given by a distinct agent within the population that specialized in the task. Here, both MATPG and MAPLE perform significantly better when using lexicase selection compared to tournament selection, highlighting the ability of lexicase selection to naturally produce elite agents for each task. Notably,

Table 2: Welch's t-test results. $\alpha_{\text{corr}} = 0.005$ with Bonferroni correction ($N = 10$). Cohen's $d > 2$ indicates a very large effect.

	p-value	Cohen's d
Tourn. MAPLE	0.000	2.68
Tourn. MATPG	0.000	2.40
Lexi. MAPLE	0.001	2.20

Table 1: Best normalized score on each obstacle, when trained for five obstacles. The normalized score is obtained by dividing the score by the average score obtained by the single obstacle training in Section 4.3. The score presented is the average score among ten seeds plus or minus the standard deviation.

Algo.	**Selection**	**Wall**	**Down**	**Maze**	**Jump**	**Stairs**
Best score by any agent within the whole population						
MAPLE	Tourn.	0.39±(0.3)	0.27±(0.2)	0.18±(0.2)	0.21±(0.1)	0.36±(0.3)
MATPG	Tourn.	0.60±(0.3)	0.17±(0.2)	0.22±(0.2)	0.23±(0.1)	0.63±(0.3)
MAPLE	Lexi.	0.73±(0.2)	0.72±(0.2)	**0.70±(0.2)**	0.22±(0.0)	0.79±(0.2)
MATPG	Lexi.	**0.81±(0.1)**	**0.94±(0.2)**	0.69±(0.2)	**0.31±(0.1)**	**0.99±(0.3)**
Score by the best agent on episodes with all obstacles						
MAPLE	Tourn.	0.24±(0.2)	0.25±(0.2)	0.12±(0.1)	0.13±(0.1)	0.21±(0.2)
MATPG	Tourn.	0.30±(0.2)	0.15±(0.2)	0.13±(0.2)	0.12±(0.1)	0.30±(0.3)
MAPLE	Lexi.	0.12±(0.2)	0.60±(0.2)	0.15±(0.2)	0.10±(0.0)	0.10±(0.1)
MATPG	Lexi.	**0.46±(0.3)**	**0.91±(0.2)**	**0.24±(0.3)**	**0.14±(0.1)**	**0.45±(0.4)**

MATPG consistently achieves equal or higher scores per obstacle than MAPLE when lexicase selection is employed.

The second part of the table evaluates the best agent from each population on the individual obstacles. The agent is selected based on performance across episodes containing all obstacles. In an ideal scenario, MATPG with lexicase selection would achieve the same results in both parts of the table. The observed discrepancy indicates that the method does not fully exploit the subpopulation generated by MAPLE. Nevertheless, even under this non-optimal condition, MATPG with lexicase selection outperforms all other approaches, including MAPLE with lexicase selection, across every obstacle. These findings suggest that lexicase selection, regardless of the underlying algorithm, can produce elite agents capable of mastering individual tasks, whereas MATPG is necessary to consolidate this ability into a single agent capable of solving multiple obstacles simultaneously.

7 Interpretability of the best agent

In this section, the interpretability of the best MATPG agent is analyzed. This agent achieves high accuracy on the *Wall*, *Down*, and *Stairs* obstacles, with normalized scores of 0.74, 0.95, and 0.77, respectively. However, its performance decreases on the *Maze* and *Jump* obstacles, where it reaches scores of 0.32 and 0.16.

To better understand its behavior, the topology of the agent is illustrated in Figure 9. The agent consists of two teams vertices, each containing three programs, and five action vertices, where each action vertex includes six programs. Five of the programs connect to the different action vertices, while the remain-

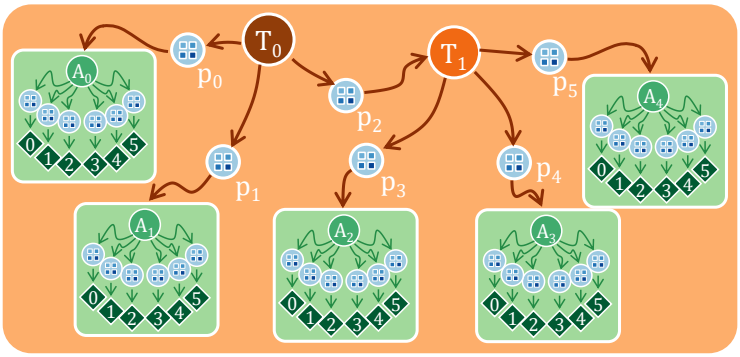


Fig. 9: Topology of the best MATPG agent on the five obstacles.

ing connect the two teams. The team vertices, action vertices, and programs are labeled in Figure 9, and the detailed behavior of the context programs is described in Algorithm 1. The activation behavior of the agent in response to specific obstacle types can be summarized as follows:

- **Jump** and **Stairs**: Under these conditions, program p_0 consistently produces the highest output value, resulting in the activation of action vertex A_0.
- **Wall**: When encountering a wall obstacle, program p_1 dominates, thereby activating action vertex A_1.
- **Down** and **Maze**: For these obstacle types, program p_2 yields the highest output, leading to the activation of team vertex T_1.
- **Maze (within T_1)**: Once the team vertex T_1 is activated, encountering a maze obstacle causes program p_4 to produce the maximum output, which in turn activates action vertex A_3.
- **Down (within T_1)**: This obstacle exhibits a more complex dynamic. When the agent is positioned far from the obstacle center, programs p_3 and p_4 alternately dominate, producing a cyclic activation pattern lasting between two and four steps per program. As the agent approaches the center of the obstacle, program p_5 becomes predominant, activating action vertex A_4.

In summary, the *Jump* and *Stairs* obstacles are handled by A_0, the *Wall* by A_1, and the *Maze* by A_3. The *Down* obstacle is addressed collaboratively by A_2, A_3, and A_4. The instructions used by each program fully explain this. In short, the traversal paths are mostly due to the state variables s_{17} and s_{18}, which correspond respectively to the obstacle identifier index and the distance to the obstacle, making the following analysis deterministic.

Algorithm 1 Programs used by the teams of the best MATPG agent, see Figure 9. Each line shows the Program p_i that is computed, depending on the state inputs s_j. The agent only used six inputs from the nineteen of the state inputs. s_3, s_6, s_7 and s_8 are inputs respectively corresponding to the angle of the back shin, the angle of the front shin, the angle of the front foot and the velocity of the x-coordinate of the front tip. s_{17} and s_{18} are additional inputs corresponding respectively to the identifier index of the obstacle and the distance to the center of the obstacle.

$p_0 \leftarrow \tan(-0.751904 \times (s_{17} + s_6))$
$p_1 \leftarrow -7.19471 \times \cos(-0.114644 \times s_8) - 6.002294 \times \frac{s_7}{s_{17}}$
$p_2 \leftarrow -0.064495 \times \log(-0.416072 \times \sin(-0.855615 \times s_6))$
$p_3 \leftarrow \tan((s_3 - \tan(s_{17})) \times 0.727467)$
$p_4 \leftarrow \tan(0.727467 \times (\tan(0.583892 \times \cos(-0.719604 \times s_{18})) + \tan(s_{17})))$
$p_5 \leftarrow \sin(0.064883 \times \cos(s_{18}))$

8 Discussion and conclusion

In this work, we designed a new MTRL benchmark to assess MATPG efficiency compared to MAPLE. Built on the *Half Cheetah* environment from MuJoCo, it features five distinct obstacles that the agent must overcome. These tasks are designed to satisfy two key properties. First, each task is mutually independent, ensuring that solving one or the original locomotion task does not provide sufficient information to solve the others. Second, all tasks remain learnable by a single MAPLE agent, establishing a baseline that verifies feasibility without requiring specialized methods. Finally, the benchmark is readily extensible, allowing future work to incorporate additional obstacles depending on experimental needs. A current limitation is that agents lack explicit obstacle structure information. Future work could investigate richer task representations by incorporating structural features to enable knowledge sharing.

We evaluate both MAPLE and MATPG under tournament selection, their default configuration, and lexicase selection, which promotes task specialization. Lexicase selection produces populations with improved coverage of task-specific behaviors in both algorithms. However, while MAPLE with lexicase evolves agents solving tasks independently, only MATPG with lexicase achieves significantly higher performance when a single agent faces all obstacles simultaneously. Analysis shows MATPG preserves the inherent interpretability of GP with clear and human-readable decision paths and obstacle-specific behaviors. Although lexicase selection performs better, it requires prior task knowledge. Developing evolutionary techniques without such requirements represents a promising future direction.

Acknowledgements

This research was funded, in whole or in part, by the Agence Nationale de la Recherche (ANR), grant ANR-22-CE25-0005-01. A CC BY license is applied to

the AAM resulting from this submission, in accordance with the open access conditions of the grant.

References

1. Amaral, R., Ianta, A., Bayer, C., Smith, R.J., Heywood, M.I.: Benchmarking genetic programming in a multi-action reinforcement learning locomotion task. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion. pp. 522–525 (2022)
2. Brameier, M.F., Banzhaf, W.: Basic concepts of linear genetic programming. Springer (2007)
3. Colas, C., Sigaud, O., Oudeyer, P.Y.: A hitchhiker's guide to statistical comparisons of reinforcement learning algorithms. arXiv preprint arXiv:1904.06979 (2019)
4. De La Torre, C., Nadizar, G., Lavinas, Y., Luga, H., Wilson, D., Cussat-Blanc, S.: Evolution of inherently interpretable visual control policies. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 358–367 (2025)
5. Desnos, K., Sourbier, N., Raumer, P.Y., Gesny, O., Pelcat, M.: Gegelati: Lightweight artificial intelligence through generic and evolvable tangled program graphs. In: Workshop on Design and Architectures for Signal and Image Processing (14th edition). pp. 35–43 (2021)
6. Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., Kavukcuoglu, K.: Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures (2018), `https://arxiv.org/abs/1802.01561`
7. Fang, Y., Li, J.: A review of tournament selection in genetic programming. In: International symposium on intelligence computation and applications. pp. 181–192. Springer (2010)
8. Fujimoto, S., van Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods (2018), `https://arxiv.org/abs/1802.09477`
9. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor (2018), `https://arxiv.org/abs/1801.01290`
10. Helmuth, T., Spector, L., Matheson, J.: Solving uncompromising problems with lexicase selection. IEEE Transactions on Evolutionary Computation **19**(5), 630–643 (2014)
11. Henderson, P., Chang, W., Shkurti, F., Hansen, J., Meger, D., Dudek, G.: Benchmark environments for multitask learning in continuous domains. CoRR **abs/1708.04352** (2017), `http://arxiv.org/abs/1708.04352`
12. Jorgensen, S., Nadizar, G., Pietropolli, G., Manzoni, L., Medvet, E., O'Reilly, U.M., Hemberg, E.: Policy search through genetic programming and llm-assisted curriculum learning. ACM Transactions on Evolutionary Learning (2025)
13. Kelly, S., Heywood, M.I.: Emergent tangled graph representations for atari game playing agents. In: Genetic Programming: 20th European Conference, EuroGP 2017, Amsterdam, The Netherlands, April 19-21, 2017, Proceedings 20. pp. 64–79. Springer (2017)
14. Kelly, S., Heywood, M.I.: Multi-task learning in atari video games with emergent tangled program graphs. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 195–202 (2017)

15. Koza, J.R., Rice, J.P.: Automatic programming of robots using genetic programming. In: AAAI. vol. 92, pp. 194–207 (1992)
16. La Cava, W., Spector, L., Danai, K.: Epsilon-lexicase selection for regression. In: Proceedings of the Genetic and Evolutionary Computation Conference 2016. pp. 741–748 (2016)
17. Mesbahi, G., Panahi, P.M., Mastikhina, O., White, M., White, A.: K-percent evaluation for lifelong rl. arXiv preprint arXiv:2404.02113 (2024)
18. Milani, S., Topin, N., Veloso, M., Fang, F.: Explainable reinforcement learning: A survey and comparative review. ACM Computing Surveys **56**(7), 1–36 (2024)
19. Miller, J., Turner, A.: Cartesian genetic programming. In: Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation. pp. 179–198 (2015)
20. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. nature **518**(7540), 529–533 (2015)
21. Nadizar, G., Medvet, E., Wilson, D.G.: Naturally interpretable control policies via graph-based genetic programming. In: European Conference on Genetic Programming (Part of EvoStar). pp. 73–89. Springer (2024)
22. Real, E., Liang, C., So, D.R., Le, Q.V.: Automl-zero: Evolving machine learning algorithms from scratch (2020), `https://arxiv.org/abs/2003.03384`
23. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017), `https://arxiv.org/abs/1707.06347`
24. Todorov, E., Erez, T., Tassa, Y.: Mujoco: A physics engine for model-based control. In: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 5026–5033 (2012). `https://doi.org/10.1109/IROS.2012.6386109`
25. Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al.: Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032 (2024)
26. Tunyasuvunakool, S., Muldal, A., Doron, Y., Liu, S., Bohez, S., Merel, J., Erez, T., Lillicrap, T., Heess, N., Tassa, Y.: dm_control: Software and tasks for continuous control. Software Impacts **6**, 100022 (2020)
27. Vacher, Q., Beuve, N., Allaire, P., Marty, T., Dardaillon, M., Desnos, K.: Hybrid genetic programming and deep reinforcement learning for low-complexity robot arm trajectory planning. In: 16th International Conference on Evolutionary Computation Theory and Applications. pp. 139–150 (2024)
28. Vacher, Q., Beuve, N., Dardaillon, M., Desnos, K.: Eurogp 2026 artifacts (2026), `https://github.com/gegelati/EUROGP-2026`
29. Vacher, Q., Kelly, S., Naqvi, A., Beuve, N., Djavaherpour, T., Dardaillon, M., Desnos, K.: Maple: Multi-action programs through linear evolution for continuous multi-action reinforcement learning. In: Proceedings of the Genetic and Evolutionary Computation Conference. pp. 1062–1071 (2025)
30. Videau, M., Ferreira Leite, A., Teytaud, O., Schoenauer, M.: Multi-Objective Genetic Programming for Explainable Reinforcement Learning. In: Lecture Notes in Computer Science. Lecture Notes in Computer Science, vol. 13223, pp. 278–293. Springer International Publishing, Madrid, Spain (Apr 2022). `https://doi.org/10.1007/978-3-031-02056-8_18`, `https://inria.hal.science/hal-03886307`
31. Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., Miao, Q.: Deep reinforcement learning: A survey. IEEE Transactions on Neural Networks and Learning Systems **35**(4), 5064–5078 (2022)
32. Welch, B.L.: The generalization of 'student's'problem when several different population varlances are involved. Biometrika **34**(1-2), 28–35 (1947)

33. Xu, Z., Wu, K., Che, Z., Tang, J., Ye, J.: Knowledge transfer in multi-task deep reinforcement learning for continuous control (2020), https://arxiv.org/abs/2010.07494